

IP-SAKTI Sahayak — MVP Overview

What was built, how it works, and why each decision was made. Written for the team.

One line: IP-SAKTI Sahayak is a decision-support copilot for Ayurveda intellectual property, scientific evidence and regulatory pathways across India, the USA and Australia. Every answer is tied to a source passage, and when the evidence is missing or conflicting, the system says so instead of guessing.

Status: MVP working end to end and deployed. Live at <https://ipsakti-three.vercel.app> (frontend on Vercel), with the backend on Render and the database on Neon (PostgreSQL + pgvector). Answers in production are generated by Google Gemini (*gemini-3.5-flash-lite*) and verified against cited sources. 69 automated tests pass.

1. The problem we are solving

Anyone bringing an Ayurveda product to market has to answer three questions at once: *Is it patentable, and what is already known? Which regulator governs it, in which country? Is the scientific evidence strong enough for the claims I want to make?* The answers sit in dozens of statutes, gazette notices, patent databases and journals, written in different languages and with different names for the same herb.

A general chatbot can answer confidently and still be wrong. In this domain a confident wrong answer is worse than no answer, because it can lead to a rejected filing or a misleading claim. So the MVP is built around **traceability and honest abstention**, not around fluent text.

1a. Why this is not a ChatGPT-style chatbot

A generic chatbot answers from its training memory. Nobody can see which source a sentence came from, and it cannot say "I don't know" reliably. For patent, regulatory and scientific questions that is a serious risk.

IP-SAKTI answers only from a curated, labelled corpus. Every claim links to a passage, is checked against it, and is labelled SUPPORTED, PARTIAL, UNSUPPORTED, CONFLICTING or INSUFFICIENT. If evidence is missing, the system abstains and explains what is missing.

The product is a workflow, not only a chat box. It has a profiler, evidence gaps, a regulatory comparison, a graph, human review and reports. Chat is one entry point into that workflow.

It is decision support, not advice. The output prepares the case for a qualified patent agent, lawyer or regulator. It never determines patentability, approval or efficacy.

1b. Who does what: users and admin

Role	What they can do
Researcher / user	Creates innovations, runs profiling, asks the assistant, reviews patents, regulatory and evidence views, sends escalations, exports reports. Sees only their own workspace.
Reviewer (workspace role)	Reviews escalations and evidence packets inside a workspace.
Admin (operator)	Runs the platform, not individual research. Manages users and workspaces, the source registry and document review (approve, reject, re-index), watches RAG and citation quality, runs evaluation, handles escalation assignment, reads the audit log, and sees system health and cost.

Who the admin is here (proposal, to be confirmed by the team): the people who operate and maintain the corpus, normally the project team itself, such as the developers and a designated curator. Admin is not a research user. It is a small, named group, and each admin account is created deliberately.

How an admin is created: with the `create_admin` command, using the `ADMIN_EMAIL` and `ADMIN_PASSWORD` environment variables. The demo admin (`admin@ipsakti.demo`) exists only for local demos and must never be used in production.

Why admin is separate: an admin can see corpus-wide quality data, so admin sign-in is a separate step with its own session. A normal user session never opens the admin console, even for an admin-role account.

2. What the MVP does — feature by feature

Area	What it does for the user	Why it is built this way
Landing page	Explains the product, shows the decision-support boundary, offers demo sign-in and the three jurisdictions.	Visitors must understand the limits before they use it. The disclaimer is shown first, not buried.
Sign in / sign up	User sign-in and sign-up pages. A Use demo account button fills the demo credentials, and the user then signs in. Admin has its own separate sign-in page, which is not linked from the public site.	Admin is a separate, server-checked area. A normal app session never grants admin access, even to an admin-role account.
Innovation Profiler	A 7-step wizard turns a formulation into structured technical features. Ambiguous names are flagged for the user to resolve.	Ayurveda names are inconsistent (Sanskrit, Hindi, botanical, chemical). Guessing silently would corrupt every later step.

AI Research Assistant	Multi-turn chat across six research modes. Each answer shows key points, evidence, status, limitations and a next step.	The structure forces the user to see the evidence and the limits, not just a paragraph.
Citation verification	Each claim is checked against the cited passage and labelled SUPPORTED, PARTIAL, UNSUPPORTED, CONFLICTING or INSUFFICIENT.	A citation that merely exists is not enough. The claim must be supported by the text it points to.
Hybrid retrieval (RAG)	Vector search plus keyword full-text search, then a deterministic reranker on relevance, jurisdiction, authority tier, freshness and document type.	Vector search finds meaning, keyword search finds exact legal terms. The reranker is rule-based so results are explainable.
Safety and abstention	Refuses unsupported jurisdictions, asks for clarification, flags prompt-injection text in queries and documents, and keeps medical and legal boundaries.	Ingested documents can contain instructions. Retrieved text is treated as data, never as commands.
Conflicts and freshness	Conflicting sources are shown side by side. Superseded documents are hidden unless asked for. Every source carries a freshness label.	Silently merging two conflicting rules would hide the very risk the user needs to see.
Scientific evidence	Study cards keep ingredient-level evidence separate from finished-formulation evidence.	A study on a single ingredient does not prove a finished product works. Mixing them is a common error.
Patents and prior art	Concept search, a feature-to-document matrix, patent-family grouping and review status.	Overlap with prior art is shown as overlap only. The system never gives a patentability verdict.
Traditional knowledge	Public or authorised context only, with access limits, attribution and an escalation path.	The system has no TKDL access and does not scrape restricted records.
Regulatory navigator	A guided questionnaire gives a provisional category per jurisdiction, its assumptions, and a side-by-side comparison.	Regulatory categories depend on facts the user may not know yet. Showing assumptions makes the uncertainty visible.
Evidence gaps and graph	Automatic gap list from LOW to CRITICAL, a risk map, and an interactive evidence graph where every edge links to its source.	The graph makes relationships visible, and provenance on each edge keeps it auditable.
Human review	Escalations to IP, regulatory or domain experts, each with an evidence packet and a status workflow.	Some decisions must be made by a qualified person. The app prepares the case, it does not close it.

Reports	A full evidence report with Markdown export and print-to-PDF.	Teams need to share findings outside the app.
Jurisdiction-aware app	The jurisdiction chosen once (India, USA, Australia) carries through patent matching, the assistant, the dashboard and admin analytics.	Before this, the selector on the landing page had no effect elsewhere. Users were misled into thinking the choice mattered.
Admin console	Users, workspaces, source registry, document review, RAG and citation monitoring, evaluation, escalations, feedback, audit log with CSV export.	Operators need to see quality and cost, and to correct the corpus, without touching the database.
Audit trail	Every key action is logged. IP addresses are stored only as salted hashes.	In a regulated domain, the team must be able to show who did what and when.

3. Technology choices and why

Layer	Choice	Reason
Frontend	Next.js 15.5 (App Router), React 19, TypeScript, Tailwind CSS	Fast to build a polished, responsive UI with one framework. Version 15.5.27 is used because the earlier 15.1.6 had a known security vulnerability, and the hosting platform refuses to deploy it.
UI animation	Framer Motion	Small, declarative animations for the carousel, count-up numbers and reveal-on-scroll.
Flow and graph	React Flow (@xyflow/react)	Interactive node-and-edge graphs with pan, zoom and click-to-inspect.
Fonts	Inter (Latin UI), Noto Sans (Devanagari for Hindi), Sora (headings), self-hosted via @fontsource	Hindi needs proper Devanagari glyphs. Self-hosting means no calls to Google Fonts at build or runtime, which keeps the app working offline and avoids third-party requests.
Backend	FastAPI, Pydantic v2, SQLAlchemy 2, Alembic	Typed request and response models, automatic API docs, and schema migrations that are reviewable. Over 100 API operations are exposed, with 4 database migrations.
Database	PostgreSQL 16 with pgvector	One database for relational data, vector search and full-text search. Fewer moving parts than a separate vector database.
Authentication	bcrypt password hashing, JWT in an httpOnly cookie, CSRF header check	Tokens are not readable by JavaScript, which reduces theft risk. The header check defends against cross-site form submissions.

LLM	Provider abstraction: Ollama (local, free), Gemini, Groq, Grok, OpenAI, DeepSeek, or a <i>mock mode</i>	The team can run without paying for an API. In mock mode, answers quote retrieved passages directly and are still verified. Switching provider is a configuration change.
Embeddings	Hashing (default, offline), OpenAI, or bge-m3	The default works with no key or internet. Better embeddings are one setting away. Known limit: hashing captures shared words, not synonyms.
Testing	pytest (69 tests); Playwright for browser checks	Tests cover access rules, citation checks, retrieval, jurisdiction filtering and the full user flow. Browser checks confirm the UI really renders.
Hosting	Vercel (frontend), Render free web service running the backend Docker image, Neon free PostgreSQL with pgvector	All three have free tiers that need no payment card, and the link is stable. The frontend calls the backend through a same-origin <code>/api</code> rewrite, so session cookies stay first-party. Render free services sleep when idle, so an UptimeRobot monitor pings the backend health endpoint to keep it awake.

4. Accessibility and government-portal style — why

Text size A-, A, A+ and high-contrast toggle. Many users, including older users and users with low vision, need larger text. The control is always visible at the top. The hero banner is laid out to fit one screen at each size, so the call to action is never pushed out of view.

Skip-to-content link, semantic landmarks, visible focus rings, 44px tap targets. These are the basic accessibility practices the public-sector guidelines expect. A full WCAG 2.0 AA audit has not been done yet.

Clear, honest footer and header notice. The site states that it is a Smart India Hackathon prototype and not an official Government of India website. This avoids implying official status, which would be misleading.

No government emblems, flags or minister photos. These would suggest official endorsement. External links to IP India, NBA, TKDL, Ministry of Ayush, CDSCO, FSSAI and WIPO are shown as reference links only, with no affiliation claimed.

5. Data and source policy — why

Label	Meaning
SEED_SUMMARY	Short excerpts or summaries of real public instruments. They must be checked against the official text before use.
DEMO_FICTIONAL	Fictional patents, guidance notes and the demo innovation. Labelled so nobody mistakes them for real filings.

PENDING_REVIEWUser uploads. Private to the workspace and unverified until reviewed.

Corpus size: 40 curated source documents across three jurisdictions. Source tiers run from 1 (official law or regulator) to 5 (user-provided, unverified). The corpus is a seed, not a live feed, so it does not yet cover every current rule.

6. Quality checks that were run

- **69 automated tests pass** (backend), including the access-rules matrix, upload validation, citation checks, jurisdiction filtering and the end-to-end flow.
- **Live browser checks** on the landing page, sign-in (user and admin), dashboard, admin console, at desktop, laptop and mobile widths, and at all three text sizes.
- **Demo login verified** through the public link: the user account signs in and reaches the app.
- **Production build** of the frontend succeeds without type errors.

7. Known limitations — stated plainly

- The corpus is a 40-document seed. Statute summaries need independent verification and the patents are fictional.
- The Gemini free tier has a daily request limit. When it is reached, answers fall back to extractive quotes (still cited and verified) until the limit resets.
- Default embeddings are offline hashing, so synonyms are weaker than with a real embedding model.
- Rate limiting is in-process, and background jobs run inline. A larger deployment needs Redis and a worker.
- Hosting runs on free tiers. The first request after a long idle period can be slow if the backend has gone to sleep.
- A full WCAG 2.0 AA accessibility audit is still pending.

8. How to run and demo it

- Local: start Postgres (Docker), then the backend (uvicorn on port 8000) and the frontend (Next.js on port 3000). Open **http://localhost:3000**.
- Demo user: **user@ipsakti.demo**, password **Demo@12345**. The "Use demo account" button fills these in.
- Admin console: open **/admin/login** directly and sign in with **admin@ipsakti.demo** and the same password. It is not linked from the public site and needs its own sign-in.
- Seeded demo innovation: *AyuCalm-X Effervescent Granules (DEMO)*, which includes an intentional ambiguity and a disease-claim flag so the safety checks can be seen working.

9. Next steps (proposed)

Immediately

- Rotate the Gemini API key used in production and update it on Render.
- After judging, change or disable the shared demo admin password (*admin@ipsakti.demo*).

Short term

- Watch the Gemini free-tier daily limit; enable billing on the API project if usage grows.
- Set a real contact address.

Medium term

- Connect a semantic embedding model, then reseed and re-run the evaluation.
- Expand the corpus and verify every seed summary against the official text.
- Run a formal accessibility audit and fix the findings.
- Add further admin accounts only when needed, using the `create_admin` command.

9a. Future scope (not built yet)

These items from the problem statement are planned for later stages. They are not part of the current MVP.

- **ABS (Access and Benefit Sharing).** Dedicated helper under the Biological Diversity Act, 2002 (as amended in 2023) and the 2024 Rules. The Act is in the corpus today, but there is no ABS workflow yet.
- **GI, trademark, design, copyright and plant-variety routing.** Separate routing and answers for these IP types. Today the corpus holds some of their sources, but no routing flow.
- **New drug category.** Classification and regulatory path for new drugs under the CDSCO new drug pathway. Not in the classifier yet.
- **Cosmetic category.** Classification and requirements under the Cosmetics Act, 1940 and Rules, 1945. Not in the classifier yet.
- **Bhashini integration.** Multilingual delivery through national language infrastructure. Today only English and Hindi are supported, inside the app.
- **Voice input and output.** Spoken questions and spoken answers. Not built.
- **DPDP Act alignment.** A documented mapping of data handling, consent, retention and audit to the Digital Personal Data Protection Act. Requires legal review before it is claimed.

10. What has been completed since the first write-up

Area	Completed
Jurisdiction	The landing page now uses a multi-select (India, USA, Australia) stored in shared app state. It carries through the dashboard, the assistant, patent matching and the research search. Patent matches for an innovation are limited to its target markets, and the admin RAG and citation views show counts by jurisdiction.

Sign-in and demo	User sign-in and admin sign-in are on separate pages. The demo user (user@ipsakti.demo) is seeded, and the "Use demo account" button fills it in. The demo admin keeps its own separate sign-in at /admin/login.
Landing page	Rebuilt in a government-portal style with a sticky accessibility bar (language, high contrast, A-/A/A+ text size), a 3-second rotating hero, feature and persona layouts, a horizontal pipeline view, a sourced industry-updates section, FAQ, and a footer. The copy is unchanged. No emblems, flags, minister photos or official claims are used.
Accessibility	Text-size and contrast settings are saved in the browser. The hero fits one screen at each size. Focus rings and skip links are in place.
App navigation	Profile dropdown at the top right, larger sidebar and navigation text, and matching sizing in the admin console.
Fonts and self-hosting	Inter, Noto Sans (with Devanagari) and Sora are served from the project itself, not from Google Fonts.
Security upgrade	Next.js upgraded from 15.1.6 to 15.5.27 to clear a known vulnerability. The production build passes.
Documentation	Technical architecture report (PS-045) prepared as PDF and HTML, with a table of contents and page numbers. The older version is kept in git history. The README was rewritten for the public repository.
Deployment	Live at https://ipsakti-three.vercel.app . Frontend on Vercel, backend on Render (production mode, secure cookies), database on Neon with pgvector, and Gemini as the answer model. All 50 pages, the API flows, the sign-in split and chat were tested on the live site.

11. Still open

- Admin access is held by the project lead for now. Further admin accounts are created only with the create_admin command.
- A real contact address is not decided. Hosting is on free tiers.
- A semantic embedding model is not connected yet (embeddings use offline hashing). Gemini is connected for answers.
- The corpus needs verification against official texts. A full accessibility audit is pending.
- Changes are not committed yet. They wait for explicit approval.